
Machine Learning Project 1:

Foundations of Classification Algorithms – Predicting Adult Incomes

Group 7: Myli Brown and Ethan Stout



PROJECT OVERVIEW & OBJECTIVES

- **Objective:** Predict whether an individual's income exceeds \$50,000 using UCI Adult ("Census Income") dataset.
 - **Data Preparation:** Handling missing values, one-hot encode categorical attributes, and standardize numeric features.
 - **Modeling:**
 - Implement Perceptron and Adaline, and Scikit-learn regularization.
 - Create a Logistic Regression and linear SVM.
 - **Analysis:** Use cross-validation for hyperparameter tuning and compare performance across multiple ML models; create visualizations of decision boundaries and analyze the impact of regularization parameter C .
 - **Reflection:** Discuss feature scaling, batch vs. Stochastic gradient descent, regularization techniques, and interpretation of C .
-

PART 1: Preprocess the Dataset

- Formatted the data frame.
- Handled missing values.
- Binned the target variable ($>50k = 1$, else=0).
- Encoded categorical features.
- Standardized numerical features (z-score).
- Split the data into X and y.
- Then further into X_train, X_test, y_train, and y_test

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week	workclass_Federal-gov	workclass_Local-gov	workclass_Private	workclass_Self-emp-inc	...	native-country_Portugal	native-country_Puerto-Rico
0	-0.413348	0.079327	1.128346	-0.147456	-0.219980	0.763915	0	1	0	0	...	0	0
1	-0.185241	-0.981089	0.344207	-0.147456	4.442817	0.763915	0	0	1	0	...	0	0
2	1.487546	0.125418	-2.008209	-0.147456	-0.219980	-0.071383	0	0	0	0	...	0	0
3	-1.325777	-0.091451	0.344207	-0.147456	-0.219980	0.429796	0	0	1	0	...	0	0
4	-0.869562	0.854670	-0.047862	-0.147456	-0.219980	-0.071383	0	0	1	0	...	0	0

5 rows × 14 columns

*Figure shows cleaned data before splitting into X and y

PART 2: Implement the Custom Perceptron

a. Train Custom Perceptron

- Implemented the custom Perceptron code provided in class.
- Tried multiple parameters and chose the model with the highest accuracy.

Parameters tested:

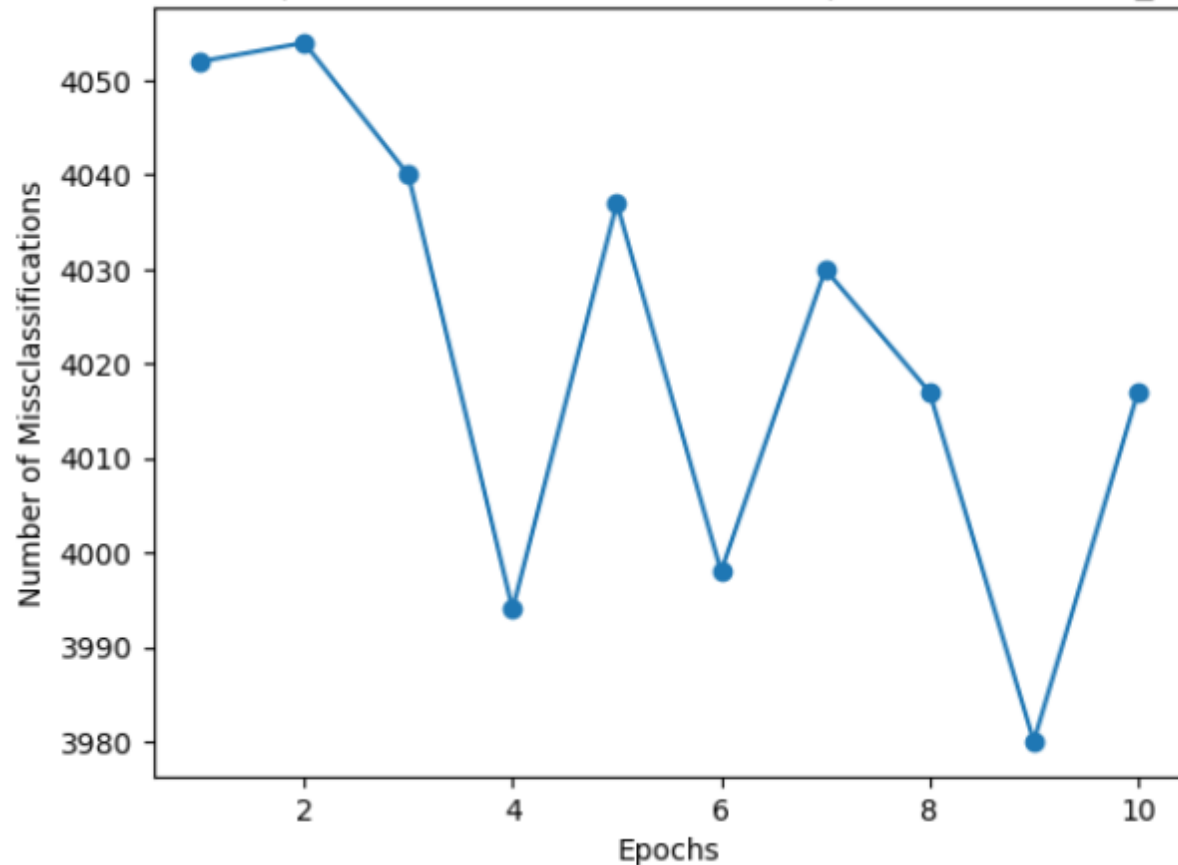
Number of iterations (n_iter): [5, 10, 15, 20, 25]

Learning rate (eta): [0.01, 0.1, 0.25, 0.5, 0.75]

```
Perceptron accuracy with n_iter 5 and eta 0.01: 0.7895
Perceptron accuracy with n_iter 5 and eta 0.1: 0.7831
Perceptron accuracy with n_iter 5 and eta 0.25: 0.7924
Perceptron accuracy with n_iter 5 and eta 0.5: 0.7773
Perceptron accuracy with n_iter 5 and eta 0.75: 0.8075
Perceptron accuracy with n_iter 10 and eta 0.01: 0.7800
Perceptron accuracy with n_iter 10 and eta 0.1: 0.8090
Perceptron accuracy with n_iter 10 and eta 0.25: 0.8106
Perceptron accuracy with n_iter 10 and eta 0.5: 0.7674
Perceptron accuracy with n_iter 10 and eta 0.75: 0.7933
Perceptron accuracy with n_iter 15 and eta 0.01: 0.7742
Perceptron accuracy with n_iter 15 and eta 0.1: 0.7868
Perceptron accuracy with n_iter 15 and eta 0.25: 0.7587
Perceptron accuracy with n_iter 15 and eta 0.5: 0.7906
Perceptron accuracy with n_iter 15 and eta 0.75: 0.7750
Perceptron accuracy with n_iter 20 and eta 0.01: 0.7454
Perceptron accuracy with n_iter 20 and eta 0.1: 0.7585
Perceptron accuracy with n_iter 20 and eta 0.25: 0.7999
Perceptron accuracy with n_iter 20 and eta 0.5: 0.7895
Perceptron accuracy with n_iter 20 and eta 0.75: 0.8030
Perceptron accuracy with n_iter 25 and eta 0.01: 0.7945
Perceptron accuracy with n_iter 25 and eta 0.1: 0.7918
Perceptron accuracy with n_iter 25 and eta 0.25: 0.7935
Perceptron accuracy with n_iter 25 and eta 0.5: 0.7761
Perceptron accuracy with n_iter 25 and eta 0.75: 0.8030
```

PART 2: Best Custom Perceptron Model

Custom Perceptron - Misclassifications over Epochs (eta=0.1, n_iter=10)



The best model had an accuracy of 0.8090

Predictions for the validation data from the best model:

```
Best Perceptron predictions (first 20):  
[0 1 1 0 0 1 1 0 0 1 0 0 0 0 0 0 1 1 1 0]
```

PART 2: Implement the Custom Adaline

a. Train custom Adaline

- Implemented the custom Adaline model code provided in class.
- Tried multiple parameters and chose the model with the highest accuracy.

Parameters tested:

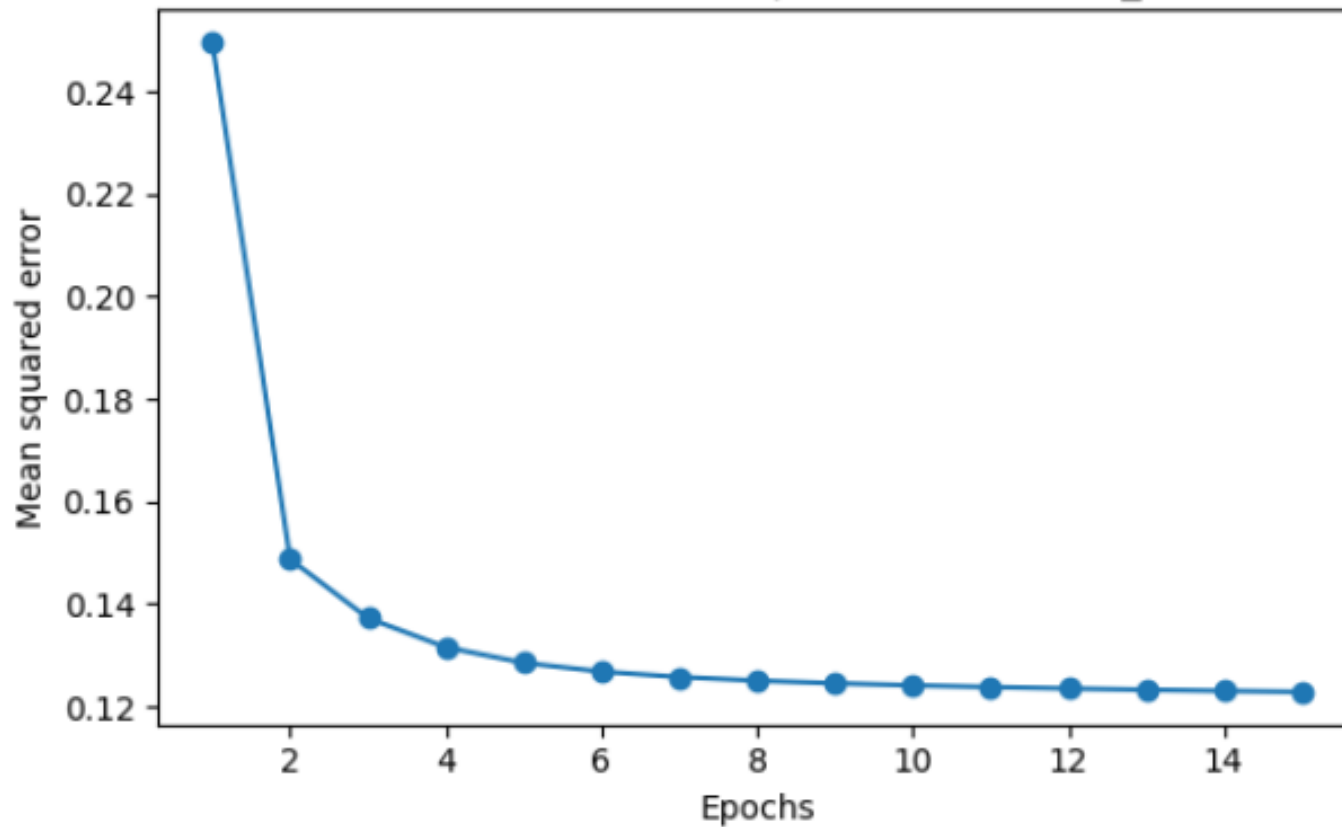
Number of iterations (n_iter): [1, 5, 10, 15, 20]

Learning rate (eta): [0.1, 0.01, 0.001, 0.0001, 0.00001]

```
Adaline accuracy with n_iter 1 and eta 0.1: 0.7649
Adaline accuracy with n_iter 1 and eta 0.01: 0.7589
Adaline accuracy with n_iter 1 and eta 0.001: 0.7589
Adaline accuracy with n_iter 1 and eta 0.0001: 0.7589
Adaline accuracy with n_iter 1 and eta 1e-05: 0.7589
Adaline accuracy with n_iter 5 and eta 0.1: 0.8200
Adaline accuracy with n_iter 5 and eta 0.01: 0.7593
Adaline accuracy with n_iter 5 and eta 0.001: 0.7589
Adaline accuracy with n_iter 5 and eta 0.0001: 0.7589
Adaline accuracy with n_iter 5 and eta 1e-05: 0.7589
Adaline accuracy with n_iter 10 and eta 0.1: 0.8291
Adaline accuracy with n_iter 10 and eta 0.01: 0.7632
Adaline accuracy with n_iter 10 and eta 0.001: 0.7589
Adaline accuracy with n_iter 10 and eta 0.0001: 0.7589
Adaline accuracy with n_iter 10 and eta 1e-05: 0.7589
Adaline accuracy with n_iter 15 and eta 0.1: 0.8309
Adaline accuracy with n_iter 15 and eta 0.01: 0.7645
Adaline accuracy with n_iter 15 and eta 0.001: 0.7589
Adaline accuracy with n_iter 15 and eta 0.0001: 0.7589
Adaline accuracy with n_iter 15 and eta 1e-05: 0.7589
Adaline accuracy with n_iter 20 and eta 0.1: 0.8311
Adaline accuracy with n_iter 20 and eta 0.01: 0.7709
Adaline accuracy with n_iter 20 and eta 0.001: 0.7589
Adaline accuracy with n_iter 20 and eta 0.0001: 0.7589
Adaline accuracy with n_iter 20 and eta 1e-05: 0.7589
```

PART 2: Best Custom Adaline Model

Custom Adaline - MSE over Epochs (eta=0.1, n_iter=15)



The best model had an accuracy of 0.8309

Predictions for the validation data from the best model:

Best Adaline predictions (first 20):

```
[0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 1 1 0 0]
```

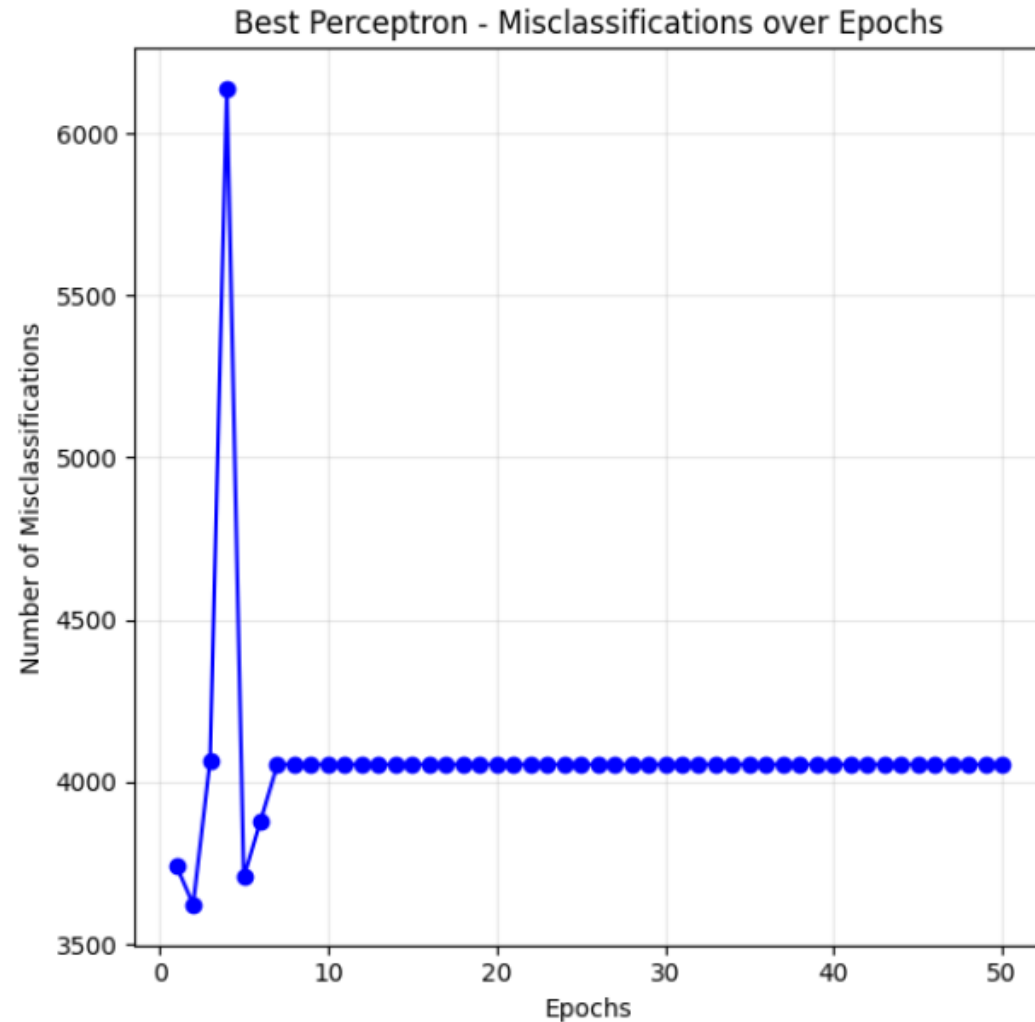
PART 2: Implement the Scikit-Learn Perceptron

```
# GridSearchCV for Perceptron
param_grid_ppn = {
    'eta0': [0.001, 0.01, 0.1, 0.5],
    'max_iter': [15, 50, 100, 200],
    'penalty': [None, 'l2', 'l1']
}

grid_ppn = GridSearchCV(
    estimator=Perceptron_sklern(
        random_state=42,
        early_stopping=True, # Stop early if no improvement
        validation_fraction=0.1, # Use 10% for validation
        n_iter_no_change=5 # Stop if no improvement for 5 iterations
    ),
    param_grid=param_grid_ppn,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_ppn.fit(X_train, y_train)
ppn_best = grid_ppn.best_estimator_
print("Best Perceptron params:", grid_ppn.best_params_)
print("Best Perceptron CV accuracy:", grid_ppn.best_score_)
```

- a. Implemented Scikit-learn's Perceptron algorithm using GridSearchCV to optimize parameters.

PART 2: Best Scikit-Learn Perceptron



Parameters for the best perceptron

Learning rate: (eta0): 0.01

Max iterations (max_iter): 15

Regularization (penalty): L1

Accuracy:

Perceptron Training Accuracy: 0.7671927554980595

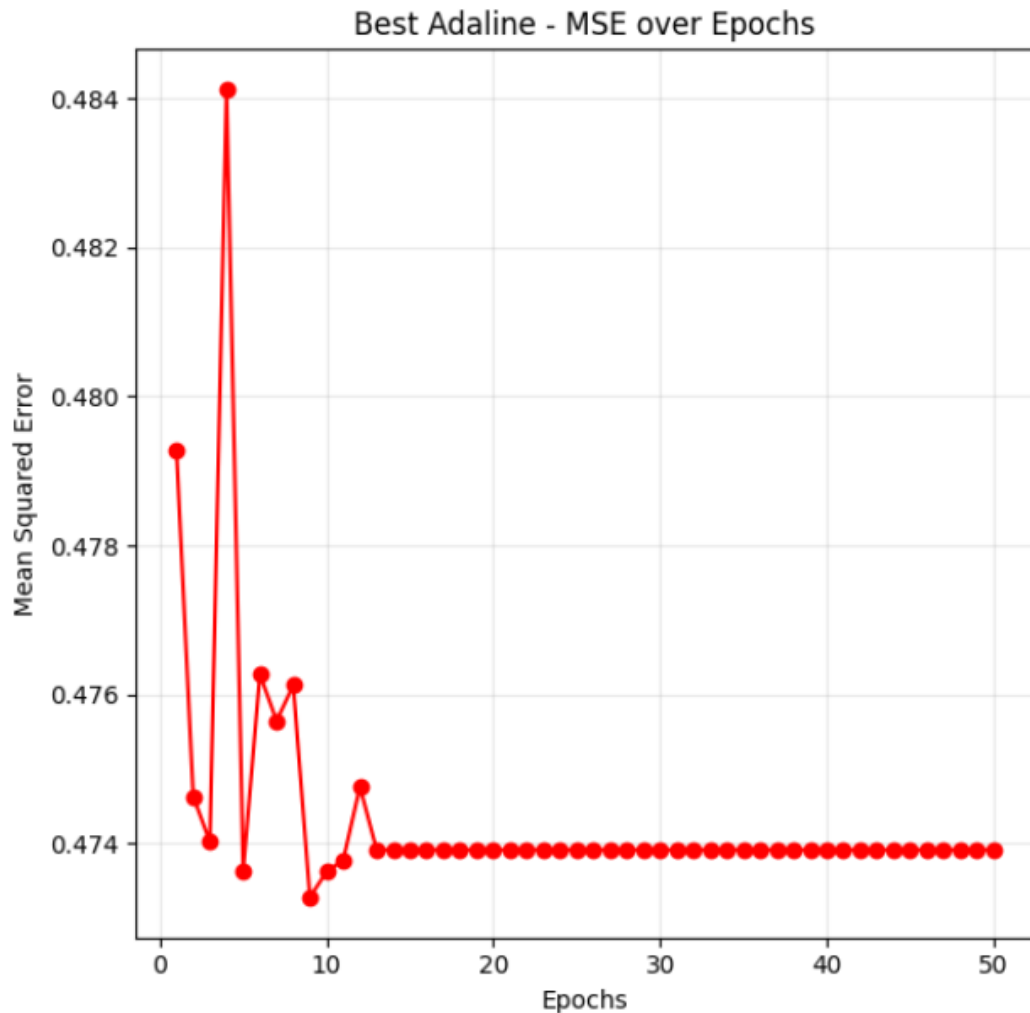
Perceptron Test Accuracy: 0.7742135761589404

PART 2: Implement the Scikit-Learn Adaline

```
28
29 # GridSearchCV for Adaline (SGD with squared loss)
30 param_grid_ada = {
31     'eta0': [0.001, 0.01, 0.1],
32     'max_iter': [15, 500, 1000, 2000],
33     'learning_rate': ['constant', 'optimal'],
34     'alpha': [0.0001, 0.001, 0.01],
35     'penalty': ['l2', 'l1', 'elasticnet']
36 }
37
38 grid_ada = GridSearchCV(
39     estimator=SGDClassifier(
40         loss='squared_error',
41         random_state=42,
42         tol=1e-4, # Tolerance for stopping criterion
43         early_stopping=True, # Enable early stopping
44         validation_fraction=0.1, # Fraction of training data to set aside for early stopping
45         n_iter_no_change=10 # Number of iterations with no improvement to wait before early stopping
46     ),
47     param_grid=param_grid_ada,
48     cv=5,
49     scoring='accuracy',
50     n_jobs=-1
51 )
52 grid_ada.fit(X_train, y_train)
53 ada_best = grid_ada.best_estimator_
54 print("Best Adaline params:", grid_ada.best_params_)
55 print("Best Adaline CV accuracy:", grid_ada.best_score_)
```

- a. Implemented Scikit-learn's Adaline algorithm using GridSearchCV to optimize parameters.

PART 2: Best Scikit-Learn Adaline



Parameters for the best Adaline

Alpha: 0.0001

Initial learning rate: (eta0): 0.001

Learning rate: constant

Max iterations (max_iter): 500

Regularization (penalty): elasticnet

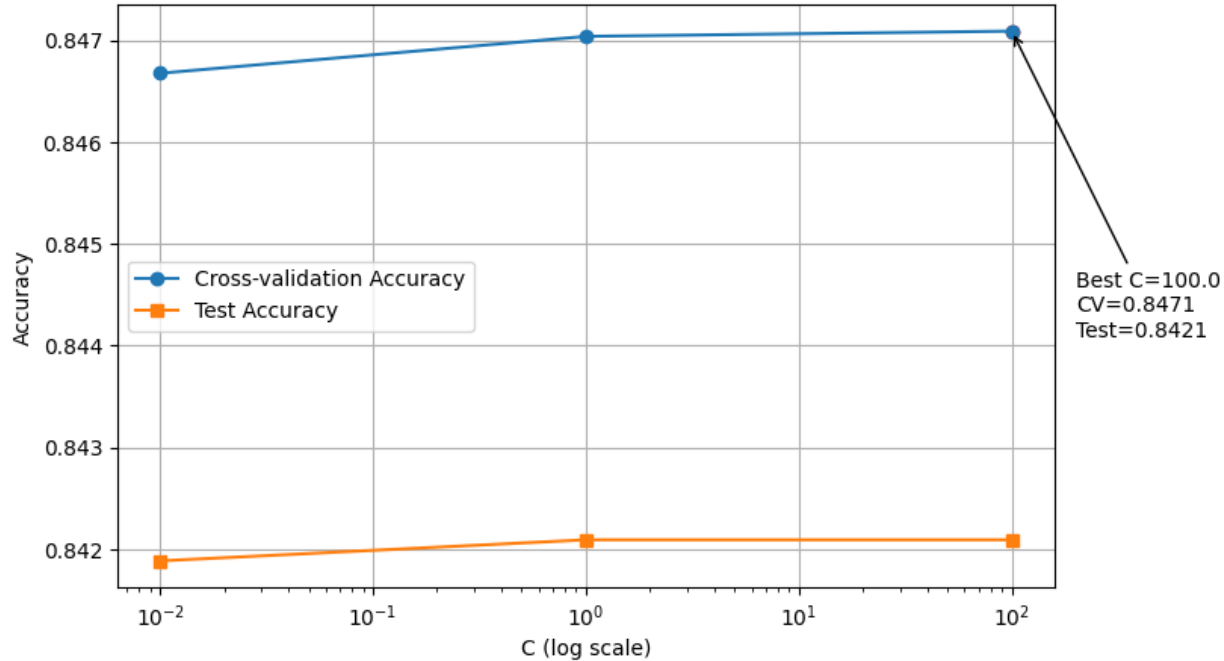
Accuracy:

Adaline Training Accuracy: 0.8341009055627425

Adaline Test Accuracy: 0.8321605960264901

PART 3: LR & SVM: IMPLEMENTATION OVERVIEW

Logistic Regression Hyperparameter Tuning



```
# Values of the regularization strength to try (smaller C = stronger regularization)
Cs = [0.01, 1.0, 100.0]

# We will store the average 5-fold CV accuracy and the hold-out test accuracy for each C
cv_scores = []
test_scores = []

# 5-fold stratified CV (keeps class balance in each fold)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for c in Cs:
    # Define the model with a given C; lbfgs is a good solver for LR on standardized data
    model = LogisticRegression(C=c, max_iter=2000, solver='lbfgs')

    # Cross-validated accuracy on the TRAINING set (fit/eval happens within folds only)
    cv_scores.append(cross_val_score(model, Xtr_std, y_train, cv=skf, scoring='accuracy').mean())

    # Refit the same model on ALL training data and evaluate once on the TEST set
    model.fit(Xtr_std, y_train)
    test_scores.append(accuracy_score(y_test, model.predict(Xte_std)))

# Pick the C that gave the highest mean CV accuracy
best_idx = np.argmax(cv_scores)
best_c = Cs[best_idx]

# Plot CV and Test accuracy vs. C (log scale makes spacing even across orders of magnitude)
plt.figure(figsize=(8,5))
plt.semilogx(Cs, cv_scores, marker='o', label='Cross-validation Accuracy')
plt.semilogx(Cs, test_scores, marker='s', label='Test Accuracy')

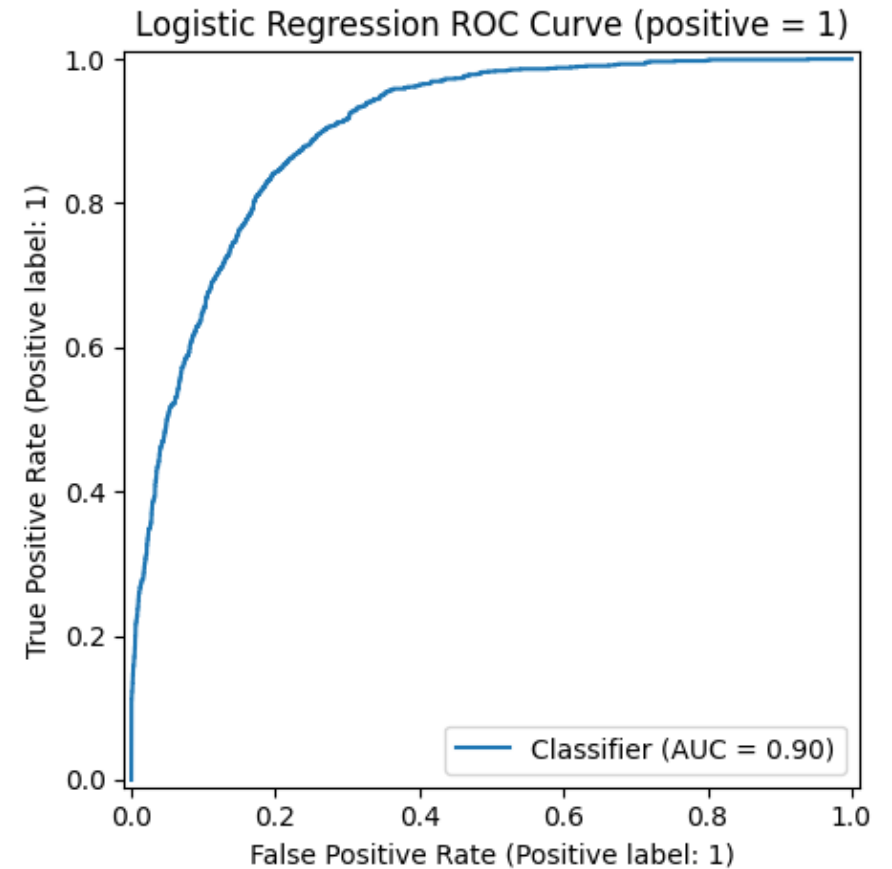
# Highlight/annotate the best C found by CV
plt.scatter(best_c, cv_scores[best_idx], color='red')
plt.annotate(
    f'Best C={best_c}\nCV={cv_scores[best_idx]:.4f}\nTest={test_scores[best_idx]:.4f}',
    xy=(best_c, cv_scores[best_idx]),
    xytext=(best_c*2, cv_scores[best_idx] - 0.003),
    arrowprops=dict(arrowstyle='>')
)

# Basic labels and styling
plt.xlabel('C (log scale)')
plt.ylabel('Accuracy')
plt.title('Logistic Regression Hyperparameter Tuning')
plt.legend()
plt.grid(True)
plt.show()
```

Python

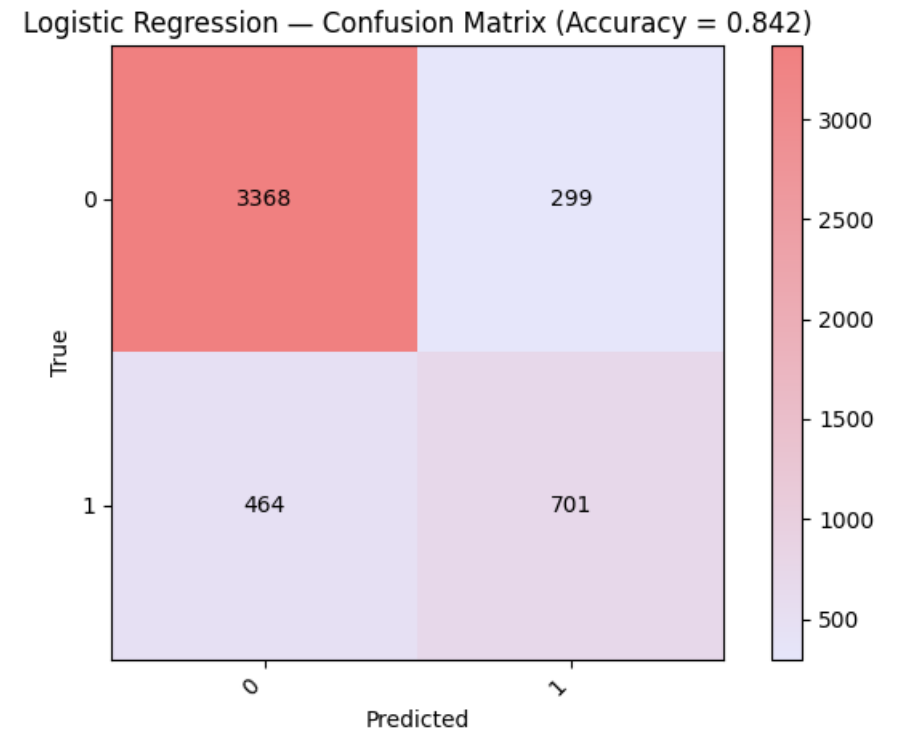
PART 3: LOGISTIC REGRESSION RESULTS SUMMARY

- Best C: 100 (5-fold stratified CV).
- Accuracy: CV ≈ 0.847 , Test ≈ 0.842 .
- “> \$50K” class: Precision ≈ 0.700 , Recall ≈ 0.600 , F1 ≈ 0.650 .
- AUC: ≈ 0.900 (strong separability).
- Note: Standardized numeric/Boolean features; refit the best model on the full train.



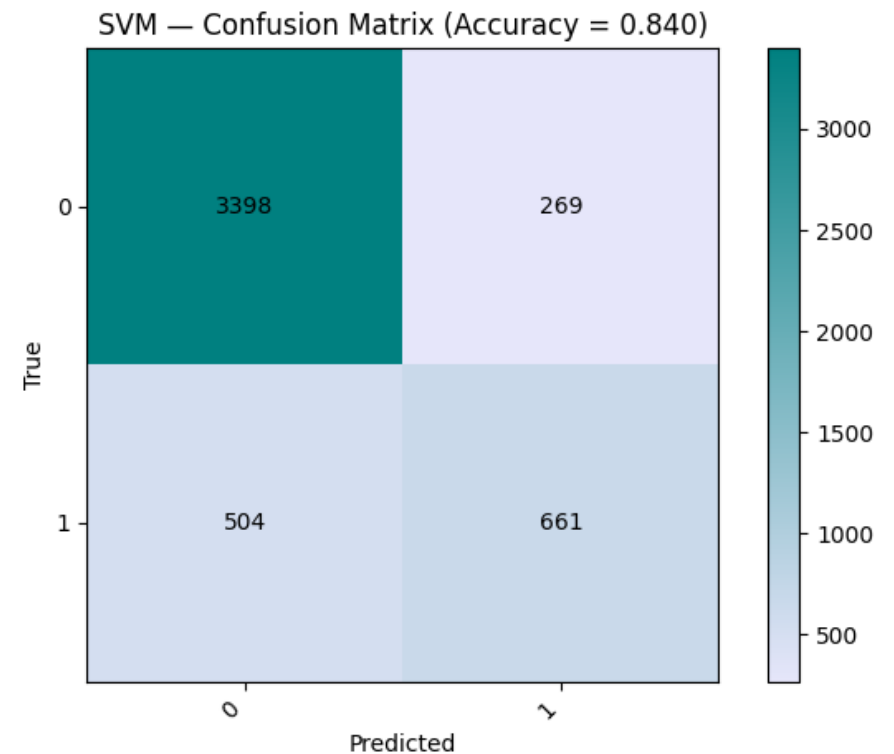
PART 3: LR CONFUSION MATRIX

- Predictions: Default threshold via `.predict()` (equivalent to 0.5 on class probability).
- Labels: Ordered as used in the plot (e.g., ["<=50K", ">50K"]).
- Accuracy (match plot title): ≈ 0.842 (rounded consistently with prior slides).



PART 3: SVM RESULTS SUMMARY

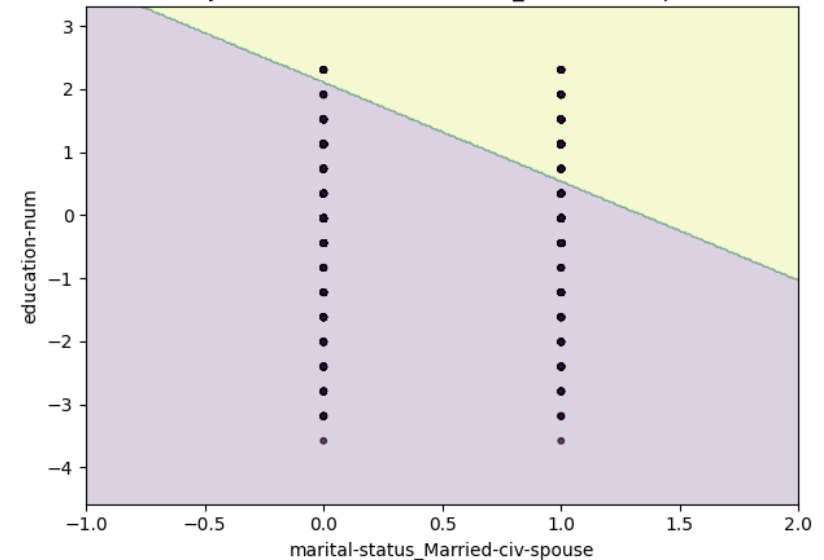
- Best C: 100 (5-fold stratified CV).
- Accuracy: CV ≈ 0.847 , Test ≈ 0.840 .
- “> \$50K” class: Precision ≈ 0.680 , Recall ≈ 0.570 , F1 ≈ 0.620 .
- Model: Linear SVM on standardized numeric/Boolean features; refit best C on full train.



PART 3: DECISION BOUNDARY

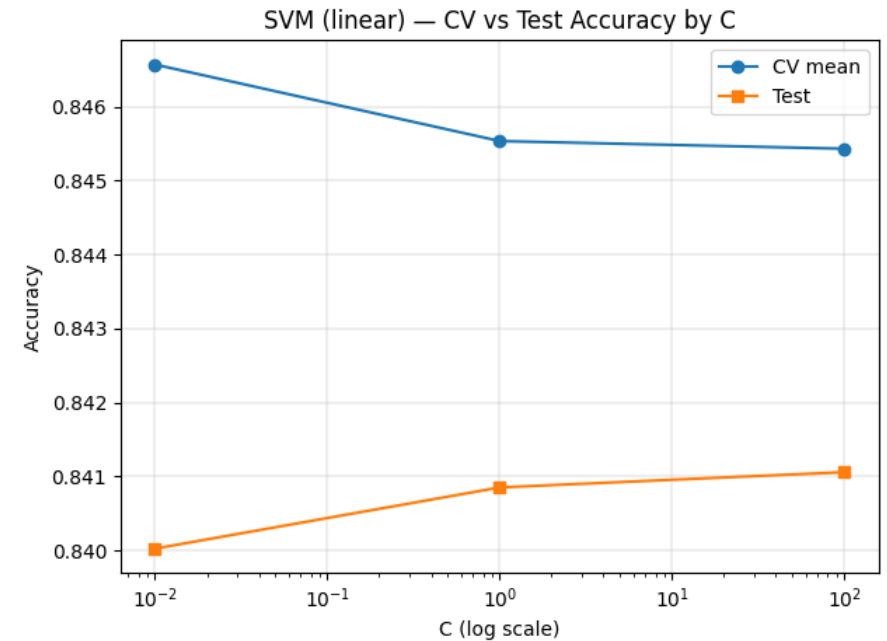
- Chosen features: marital-status_Married-civ-spouse vs education-num.
- LR vs SVM: LR gives a smooth probability boundary; SVM maximizes margin using support vectors.
- $C = 100$: both selected via 5-fold CV; higher $C \rightarrow$ tighter fit, narrower margin.

SVM Decision Boundary — Features: marital-status_Married-civ-spouse & education-num



PART 3: CV VS TEST ACCURACY BY C

- Best C: 100 by mean CV accuracy.
- Accuracy: CV ≈ 0.847 , Test ≈ 0.842 (curves closely aligned).
- Interpretation: Smaller C = stronger regularization (simpler model); larger C = weaker regularization (tighter fit).



PART 3: LR VS SVM – COMBINED COMPARISON

Logistic Regression

- Best C: 100 (5-fold stratified CV)
- Accuracy: CV ≈ 0.847 , Test ≈ 0.842
- “> \$50K” class: Precision ≈ 0.700 , Recall ≈ 0.600 , F1 ≈ 0.650
- AUC: ≈ 0.900

Linear SVM

- Best C: 100 (5-fold stratified CV)
 - Accuracy: CV ≈ 0.847 , Test ≈ 0.840
 - “> \$50K” class: Precision ≈ 0.680 , Recall ≈ 0.570 , F1 ≈ 0.620
-

PART 4: Reflection and Conceptual Questions

Why is feature scaling important for gradient-based algorithms?

- **Prevents one feature from dominating the gradient**
 - If features are on very different scales (income in the thousands vs. age in tens), the feature with the larger range will disproportionately affect the gradient updates.
 - **Ensures fairness across features**
 - In regularization, scaling ensures all features are penalized fairly, instead of heavily penalizing features with larger magnitudes regardless of their importance.
 - **Speeds up convergence**
 - Gradient descent is able to take smoother, more direct steps toward the minimum.
 - **Stabilizes learning rates**
 - With scaling, you can use a higher learning rate and are less likely to overshoot, which improves efficiency.
-

PART 4: Reflection and Conceptual Questions

Explain the difference between batch gradient descent and stochastic gradient descent.

Batch Gradient Descent

- Looks at all the training data at once to compute the gradient.
- After running through the full data, it makes one update per epoch.
- **Pros:** Very stable updates. Will converge smoothly toward the minimum.
- **Cons:** Slow on large datasets.

Stochastic Gradient Descent

- Looks at one random training example at a time to update the weights.
 - Makes one update per data point for each epoch.
 - **Pros:** Much faster for large datasets.
 - **Cons:** Doesn't converge as smoothly as Batch Gradient Descent.
-

PART 4: Reflection and Conceptual Questions

Why does scikit-learn Perceptron and Adaline algorithms outperform book code?

Scikit-learn implementations of Perceptron and Adaline tend to outperform manual implementations because:

- The scikit-learn module is well tested and perfected.
- It is usually quicker than manually built algorithms.
- It has better default optimization details like shuffling, stopping criteria, and regularization.

A custom implementation can become as good as the scikit-learn models if they implement the same optimizations correctly, which is difficult and easy to get subtly wrong.

PART 4: Reflection and Conceptual Questions

Compare the decision boundaries of logistic regression and support vector machine.

Logistic regression creates a straight line, plane, or hyperplane that best separates the classes by modeling probabilities. Logistic regression focuses on probabilities and shifts the decision boundary based on every point in the dataset.

Support vector machine looks to find the widest margin between classes, not looking at probabilities at all. Support vector machine uses only a handful of critical points (support vectors) to update the decision boundary.

PART 4: Reflection and Conceptual Questions

What is the role of regularization in preventing overfitting?

- **Overfitting** happens when a model learns the noise in the training data instead of finding the underlying pattern. This results in a high training accuracy, but poor performance on testing data.
- **Regularization** adds a penalty to the model for being too complex. It can help to shrink the weights for features or even remove some features entirely.

Regularization prevents overfitting by keeping the model simpler, helping it capture the true signal rather than memorizing the noise.

PART 4: Reflection and Conceptual Questions

Vary the C values of the scikit-learn Logistic Regression and linear SVC models with $[0.01, 1.0, 100]$. Discuss the impact.

- What C does: In scikit-learn, $C = 1/\lambda \rightarrow$ smaller C = stronger regularization, larger C = weaker regularization (applies to LR & SVM).
 - Logistic Regression: Best $C = 100 \rightarrow CV \approx 0.847$, Test ≈ 0.842 ; CV and Test closely aligned $\rightarrow$ good generalization.
 - Linear SVM: Best $C = 0.01 \rightarrow CV \approx 0.8466$, Test ≈ 0.840 ; stronger regularization (wider margin) generalized better on this high-dimensional one-hot data.
 - Interpretation:
 - LR benefited from weaker regularization (higher C) — coefficients could fit more signal without overfitting.
 - SVM benefited from stronger regularization (lower C) — wider margin reduced overfitting/noise sensitivity.
 - Practical takeaway: Tune C via CV, watch CV–Test alignment; scale features; then choose the model/threshold based on precision–recall needs.
-